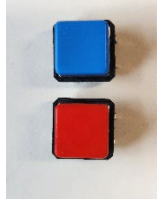


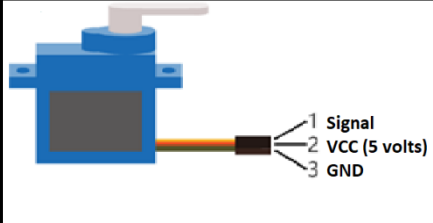
Travail pratique 3_3

Contrôle d'un servo et d'un afficheur avec un interface graphique

On manipulera ici un servo et un afficheur graphique à l'aide d'une interface graphique en Python. Le servo pourra aussi être contrôlé par deux boutons. L'utilisation de la bibliothèque RPi.GPIO n'est pas permise. On aura besoin des quatre composants suivants:

Un Servo <i>sur lequel on va exercer sur son bras un contrôle en le faisant tourner</i>	Un afficheur à sept segments <i>pour afficher l'état du servo et faire défiler les lettres entrées dans une boîte de saisie</i>	Deux boutons <i>pour contrôler le servo</i>
		

Servo (Tower Pro sg90): un servomoteur (souvent abrégé en « servo », provenant du latin *servus* qui signifie « esclave ») est un moteur capable de maintenir une opposition à un effort statique et dont la position est vérifiée en continu et corrigée en fonction de la mesure¹. Le bras du servo peut effectuer des rotations de 180 degrés. Une seule broche (fil orange) suffit pour exercer un contrôle à travers un signal:

	Largeur d'impulsion	Angle du servo	<i>min_pulse_width</i> à l'angle minimal <i>max_pulse_width</i> à l'angle maximal
	0.5ms	0 degré	
	1ms	45 degrés	
	1.5ms	90 degrés	
	2ms	135 degrés	
	2.5ms	180 degrés	

C'est votre programme Python qui aura la charge d'effectuer ces rotations. Les rotations seront par tranche de ± 45 degrés. Voici les informations nécessaires pour bien faire fonctionner votre servo:

1. Fixez le bras sur le dessus du servo:



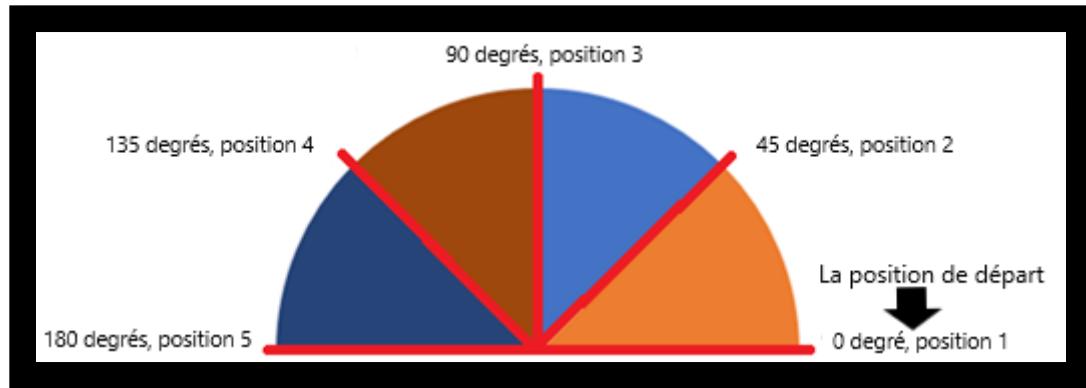
Le bras montré est à +180 degrés: `servo.angle = 180`

2. Branchez le servo à votre RasPi.
3. Utilisez la classe **AngularServo**² dans la bibliothèque **gpiozero** pour contrôler votre servo. Vous allez préciser au moins six arguments dans votre constructeur :
 - i. le numéro de la broche de contrôle

¹ <https://fr.wikipedia.org/wiki/Servomoteur>

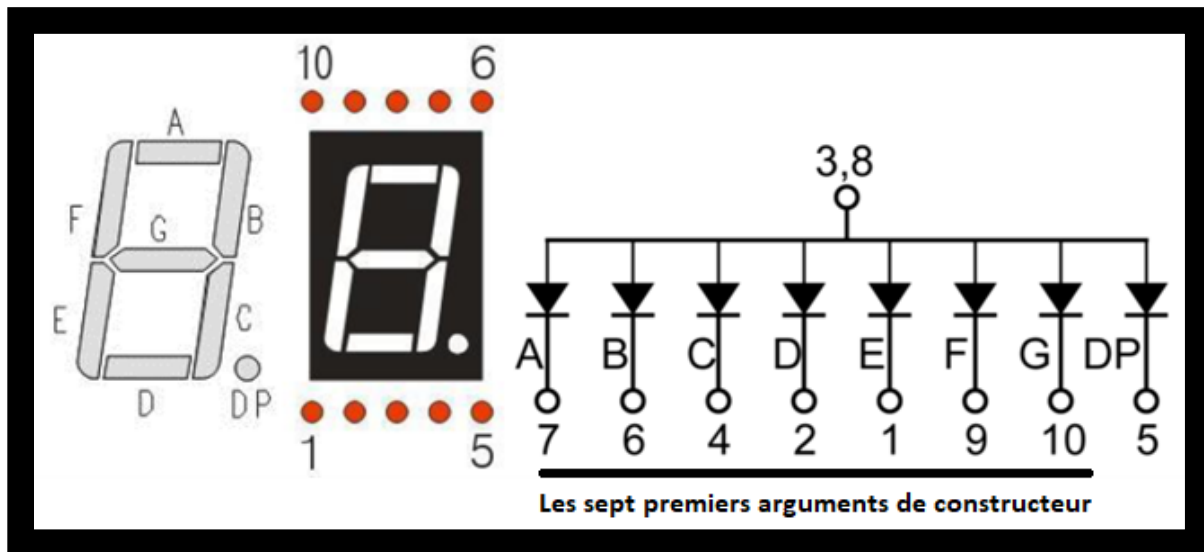
² https://gpiozero.readthedocs.io/en/latest/api_output.html#angularservo

- ii. l'angle de départ, fixé à zéro
 - iii. les angles minimum et maximum, en degrés
 - iv. les largeurs d'impulsion minimale et maximale du servo, en secondes
4. Ignorez l'avertissement suivant :
- `/usr/lib/python3/dist-packages/gpiozero/output_devices.py:1532: PWMSoftwareFallback: To reduce servo jitter, use the pigpio pin factory. See https://gpiozero.readthedocs.io/en/stable/api\_output.html#servo for more info`
`warnings.warn(PWMSoftwareFallback(`
5. Faites les ajustements pour que les angles du bras de votre servo soient comme dans l'illustration suivante :



Le numéro de la position est celui affiché sur l'afficheur

L'afficheur à sept segments: Un affichage à sept segments est une technique d'affichage basé sur sept segments qui peuvent être activés ou désactivés en fonction du motif graphique à produire³. L'afficheur que vous utiliserez possède les branchements suivants:



Par exemple, le segment A est activé par la broche 7. La broche 5 sert à afficher un point décimal (DP) au besoin. Cet afficheur est à anode commune (les broches 3 et 8 sont donc connectées à 3.3 volts); autrement dit, si on veut qu'un segment s'allume, il faut lui envoyer un signal BAS (il y a un paramètre dans le constructeur pour spécifier ce comportement). Vous pourriez contrôler vous-même les segments à allumer, mais vous pouvez utiliser une classe de **gpiozero** qui s'occupera de le faire pour vous: **LEDCharDisplay**⁴. Notez bien, ce sont des LEDS, elles ont chacune besoin d'une résistance!

³https://fr.wikipedia.org/wiki/Affichage_à_sept_segments

⁴<https://gpiozero.readthedocs.io/en/stable/recipes.html#ledchardisplay>. Si cette classe n'est pas incluse avec le SE que vous avez installé sur le Raspberry Pi, il faudra refaire l'installation comme suit:

```
sudo apt remove python3-gpiozero
sudo pip3 install gpiozero
```

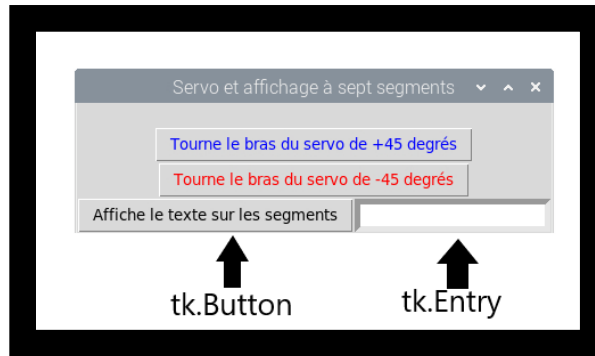
En testant cette classe, vous allez remarquer que les lettres [K, M, V, W, X, Z] provoquent une exception puisqu'elles n'existent pas dans la police par défaut. Vous devez donc créer une nouvelle police en utilisant la classe **LEDCharFont**⁵ et l'intégrer à votre afficheur pour obtenir tous les caractères suivants, en plus des chiffres de 0 à 9 :



Deux boutons (rouge et bleu): ces boutons serviront à augmenter ou diminuer l'angle du bras du servo :

1. **Bouton bleu:** tourne le bras du servo de +45 degrés. Si le bras est déjà à sa position maximale de 180 degrés, on le ramène à sa position minimale. Il faut aussi indiquer la position finale sur l'afficheur (entre 1 et 5). *Cette fonctionnalité sera aussi disponible via l'interface graphique.*
2. **Bouton rouge:** tourne le bras du servo de -45 degrés. Si le bras est déjà à sa position minimale de 0 degré, on le ramène à sa position maximale. Il faut aussi indiquer la position finale sur l'afficheur (entre 1 et 5). *Cette fonctionnalité sera aussi disponible via l'interface graphique.*

Interface graphique: votre programme s'exécutera à travers une interface graphique ayant l'allure suivante:



Votre interface devra être écrite en (**gpiozero**) Python avec la bibliothèque graphique de votre choix. Elle fournira les fonctionnalités suivantes à l'aide de trois boutons et une boîte de texte:

1. **Tourne le bras du servo de +45 degrés:** même fonctionnalité que le bouton bleu.
2. **Tourne le bras du servo de -45 degrés:** même fonctionnalité que le bouton rouge.
3. **Affiche le texte sur les segments:** affichera le mot écrit dans la boîte, une lettre par seconde. Votre programme ne permettra pas d'afficher un mot avec un caractère qui n'est pas dans votre nouvelle police. Une fois la dernière lettre du mot écrite, le segment affichera la position du bras du servo.

Si vous utilisez *tkinter*, la séance six (6) vous fournira l'essentiel dont vous avez besoin. Notez que si votre approche nécessite de gérer des événements dans un thread, alors le squelette suivant pourra vous aider :

⁵ https://gpiozero.readthedocs.io/en/latest/api_boards.html#ledcharfont

```
def faitCeci():
    votreCode

def autre_tacheAvecEvenement():
    composant.evenement = faitCeci
    pause() # la pause doit être ici, pas dans le programme principal!

if __name__ == "__main__":
    autre_tache_thread = threading.Thread(target = autre_tacheAvecEvenement)
```

Remise : vous devrez montrer votre circuit fonctionnel à votre enseignant vendredi le 3 novembre et remettre sur LÉA le programme Python ainsi que le schéma ou la platine conçue sur Fritzing le même jour avant minuit. C'est un travail individuel.

Barème de correction :

Fonctionnalités	Composants					Points
	Servo	Afficheur	Bouton bleu	Bouton rouge	Interface graphique	
Position minimale de départ à 0°	4					4
Position maximale à 180°	4					4
Mouvement de +45° normal	4		6		4	14
Mouvement de +45° extrême	4		6		4	14
Mouvement de -45° normal	4			6	4	14
Mouvement de -45° extrême	4			6	4	14
Affichage de la position		8				8
Affichage d'un mot invalide		2			2	4
Affichage d'un mot valide et retour à la position		8			6	14
Nouvelle police		6			4	10
Points	24	24	12	12	28	100

Remise LÉA :

- programme Python - 10
- circuit Fritzing - 10